

Prompt 3

KKC TEXT ADVENTURE — PROMPT 3 OF 180

=====

Sympathy System: Adjudication, Materials, Cost, and Consequence

=====

This prompt builds directly on Prompts 1 and 2.

Do not restructure, rename, or refactor anything from those prompts.

Only add to what exists.

HARD BANS — UNCHANGED

No external APIs. No HTTP clients. No LLM. No Anthropic, OpenAI, or any remote service. No web server. No frontend. Fully offline only.

No new abstractions unless explicitly required here.

No new files unless listed in this prompt.

Do not refactor working code from earlier prompts.

CANON RULES FOR SYMPATHY — READ FIRST

Sympathy in the books operates on three confirmed laws:

Doctrine of Correspondence: similarity enhances sympathy.

The more two objects resemble each other, the stronger the sympathetic link between them.

Principle of Consanguinity: a piece of a thing can represent the whole of a thing.

Law of Conservation: energy cannot be created or destroyed.

The sympathist redirects energy — they do not create it.

The cost is always paid by something. Usually the practitioner's own body heat.

Further confirmed canon facts:

- The Alar is the practitioner's ability to hold a belief firmly while simultaneously holding its contradiction. It is not willpower in a generic sense — it is a specific split-attention skill that must be trained and can be degraded.
- A sympathetic link requires: the Alar held firm, at least two objects to bind, and sufficient similarity or consanguinity between them.

- Every link has a efficiency rating — more similar objects lose less energy. Dissimilar objects waste most of the energy as heat on the practitioner.
- Body heat is the primary cost. Cold, injured, or exhausted practitioners cannot sustain links safely.
- Failure states: the link slips (energy lost, nothing achieved), bleed-through (practitioner feels what the bound object feels), or a full sympathetic backlash (injury or unconsciousness).
- Sympathy cannot be used reliably under high emotional distress without exceptional training. Kvothe is talented but not yet trained beyond basics at the start of University.
- Naming is NOT sympathy. Do not conflate them. Naming is out of scope for this prompt entirely.

DO NOT:

- Invent new sympathy laws not in canon
- Give sympathy a mana bar or cooldown
- Make sympathy reliably safe in early University era
- Allow advanced bindings (multiple simultaneous links on different objects) for an early E'lor — the books do not support this at this stage
- Narrate outcomes with confident invented detail beyond what the engine calculates

GOAL OF THIS PROMPT

By the end of Prompt 3, the player should be able to:

- Type `use sympathy` or `bind [object] to [object]` and have the engine adjudicate whether the attempt is valid, at what cost, and with what risk
- Feel the physical cost of a successful binding as warmth leaving their body (reflected in player state)
- Fail a binding and feel the consequence — a slip, bleed-through, or backlash depending on circumstances
- Buy sympathy materials at the Fishery or find them in inventory
- Track alar strength as a real degradable resource
- Understand through play that sympathy is dangerous, costly, and requires preparation — not a casual ability

SCOPE OF THIS PROMPT

This prompt covers:

1. SympathyState added to PlayerState

2. Sympathy materials tracked in inventory
3. Sympathy adjudication engine (sympathyEngine.ts)
4. `use sympathy` / `bind` command wired into actions.ts
5. Buy materials command at university_fishery_outer
6. Warmth/cold tracking added to PlayerState
7. Local narration templates for sympathy outcomes
8. Tests for adjudication logic

This prompt does NOT cover:

- Sygaldry (Prompt 7)
- Naming (never a player system in early era)
- Advanced multi-link bindings
- Kilvin as a fully modelled NPC (Prompt 4)
- Sympathy used in combat or dramatic events (later canon events)

NEW FILES IN THIS PROMPT

Add exactly these files:

```
src/  
  engine/  
    sympathyEngine.ts ← adjudication, cost, failure logic  
  content/  
    sympathyNarration.ts ← local authored outcome text pools  
tests/  
  sympathyEngine.test.ts
```

Modify these existing files:

```
src/types/index.ts      ← add SympathyState, WarmthLevel,  
                        SympathyAttempt, SympathyResult  
src/engine/state.ts     ← add sympathy_state and warmth to  
                        PlayerState and initDefaultPlayerState  
src/engine/actions.ts   ← add sympathy commands and buy materials  
src/db/schema.ts        ← no changes needed  
src/db/seed.ts          ← no changes needed
```

TYPES — additions to src/types/index.ts

Add these. Do not remove or change existing interfaces.

```
export type WarmthLevel =  
  'warm' | 'comfortable' | 'cool' | 'cold' | 'dangerously_cold';
```

```
export type SympathyOutcome =
```

```
'success' | 'slip' | 'bleedthrough' | 'backlash' | 'blocked';
```

```
export interface SympathyState {  
  alar_strength: number;    // 0–100. 100=fully focused, 0=broken  
  warmth: number;          // 0–100. 100=warm, 0=dangerously cold  
  active_bindings: number;  // how many links currently held (0 for MVP)  
  times_used_today: number; // resets on sleep  
}
```

```
export interface SympathyAttempt {  
  source_item_id: string;  // item in inventory being used as source  
  target_item_id: string;  // item being affected  
  intent: string;          // plain description: e.g. 'lift target'  
}
```

```
export interface SympathyResult {  
  outcome: SympathyOutcome;  
  heat_cost: number;      // warmth points lost (0 if blocked/slip)  
  alar_cost: number;      // alar_strength points lost  
  injury: string | null;   // null unless backlash  
  narration_key: string;  // key into sympathyNarration.ts  
  state_changes: Partial<SympathyState>;  
}
```

Also add to PlayerState:

```
sympathy_state: SympathyState;  
warmth: number; // 0–100, mirrors sympathy_state.warmth for  
              // easy access in existing rendering functions
```

Update initDefaultPlayerState() in engine/state.ts:

```
sympathy_state: {  
  alar_strength: 60,    // E'lor baseline — trained but not expert  
  warmth: 80,          // comfortable to start  
  active_bindings: 0,  
  times_used_today: 0,  
},  
warmth: 80,
```

Also update initDefaultPlayerState() inventory to include the sympathy kit items that should be present at start:

```
{  
  id: 'candle_tallow',  
  name: 'tallow candle',  
  quantity: 3,  
  notes: 'Basic sympathetic binding material.'  
},  
{  
  id: 'wicking',
```

```
name: 'wicking cord',
quantity: 1,
notes: 'For use in sympathy preparations.'
}
```

SYMPATHY MATERIALS

The following items are valid sympathy materials.
These are the only items that can be used as source or target in a sympathetic binding at E'lir level. Do not expand this list.

candle_tallow	— common, low similarity bonus
wicking	— component, low similarity bonus
iron_drab	— a coin, excellent for metal-to-metal bindings
clay_lump	— modest similarity to organic matter
pine_pitch	— adhesive, aids concentration (Ben's trick) does not improve the link itself
wax_stub	— melted candle remnant, basic use

These can be purchased at university_fishery_outer.

Prices (canon-consistent with student budgets):

candle_tallow	2 drabs each
wicking	3 drabs per length
iron_drab	1 drab (it is a coin — it costs itself)
clay_lump	2 drabs
pine_pitch	4 drabs per small pot
wax_stub	1 drab each

The Fishery outer workspace is the only place to buy these in MVP.
The player must be at university_fishery_outer to purchase.

ENGINE — src/engine/sympathyEngine.ts

This is the core adjudication module. It owns all sympathy truth.
It does not narrate. It does not call narrator functions.
It returns a SympathyResult that actions.ts passes to narration.

Implement:

```
getSimilarityScore(  
  source_item_id: string,  
  target_item_id: string
```

): number

Returns a similarity score 0.0–1.0 based on these rules:

Same item id:	1.0 (identical)
Both are metal items:	0.7 (e.g. two iron_drabs)
Both are wax/candle items:	0.65 (candle_tallow + wax_stub)
One wax, one cord:	0.3
One metal, one organic:	0.15
Otherwise:	0.1

These scores must be defined as a lookup table or switch — not inferred by the LLM at runtime. Codex writes it; it stays fixed.

```
calculateHeatCost(  
  similarity: number,  
  alar_strength: number  
) : number
```

Returns warmth points the player will lose.

Formula (all integers):

```
base_cost = 15  
efficiency_reduction = Math.floor(similarity * 10)  
alar_modifier = alar_strength < 40 ? 5 : 0 // weak alar = more cost  
total = base_cost - efficiency_reduction + alar_modifier
```

Clamp result between 3 and 25.

```
calculateRisk(  
  state: PlayerState,  
  similarity: number  
) : number
```

Returns a risk score 0–100 representing chance of failure.

```
base_risk = 20  
if state.warmth < 30: base_risk += 30  
if state.warmth < 50: base_risk += 15  
if state.fatigue > 70: base_risk += 15  
if state.hunger > 70: base_risk += 10  
if state.sympathy_state.alar_strength < 30: base_risk += 25  
if state.sympathy_state.times_used_today >= 3: base_risk += 20
```

```
if similarity < 0.2:      base_risk += 20
if similarity >= 0.6:    base_risk -= 10
```

Clamp between 5 and 95.

```
resolveOutcome(
    risk: number,
    state: PlayerState
): SympathyOutcome
```

Determine outcome deterministically from risk and state.
Do NOT use Math.random(). Use this deterministic formula:

```
seed = (state.sympathy_state.times_used_today * 17
        + state.day_number * 7
        + Math.floor(state.warmth)
        + Math.floor(state.sympathy_state.alar_strength)) % 100
```

```
if seed >= risk:      return 'success'
if seed >= risk - 20: return 'slip'
if seed >= risk - 40: return 'bleedthrough'
return 'backlash'
```

This is deterministic. The same state always produces the same outcome. The player can learn the system through play and preparation.

```
adjudicate(
    attempt: SympathyAttempt,
    state: PlayerState
): SympathyResult
```

The main function. Chains the above together.

Step 1 — Block invalid attempts:

```
If source_item_id not in player inventory:
    return blocked, narration_key: 'no_source_material'
If target_item_id not in VALID_SYMPATHY_MATERIALS and
not in player inventory:
    return blocked, narration_key: 'invalid_target'
If state.sympathy_state.alar_strength < 10:
    return blocked, narration_key: 'alar_broken'
If state.warmth < 10:
    return blocked, narration_key: 'too_cold'
```

Step 2 — Calculate:

```
similarity = getSimilarityScore(source, target)
heat_cost = calculateHeatCost(similarity, alar_strength)
risk      = calculateRisk(state, similarity)
outcome   = resolveOutcome(risk, state)
```

Step 3 — Build state_changes based on outcome:

On success:

```
warmth reduced by heat_cost
alar_strength reduced by 5
times_used_today incremented by 1
injury: null
narration_key: 'success'
```

On slip:

```
warmth reduced by Math.floor(heat_cost / 2)
alar_strength reduced by 10
times_used_today incremented by 1
injury: null
narration_key: 'slip'
```

On bleedthrough:

```
warmth reduced by heat_cost
alar_strength reduced by 15
times_used_today incremented by 1
injury: 'minor sympathetic bleedthrough — sensation in hands'
narration_key: 'bleedthrough'
```

On backlash:

```
warmth reduced by heat_cost * 2 (clamped at making warmth = 0
if it would go negative)
alar_strength reduced by 25 (min 0)
times_used_today incremented by 1
injury: 'sympathetic backlash — muscle spasm, pain in chest'
narration_key: 'backlash'
```

Step 4 — Return SympathyResult with all fields populated.

adjudicate() does not mutate PlayerState. It returns a result.
The caller in actions.ts applies state_changes to PlayerState.

```
applySympathyResult(
  state: PlayerState,
  result: SympathyResult
): PlayerState
```

Applies result.state_changes to a copy of PlayerState.
Returns the new state without mutating the input.

Updates:

- sympathy_state.warmth
- sympathy_state.alar_strength
- sympathy_state.times_used_today
- warmth (top-level mirror)
- injuries array — if result.injury is not null, append it

```
buyMaterial(  
  item_id: string,  
  quantity: number,  
  state: PlayerState  
): { newState: PlayerState; message: string }
```

Checks player is at university_fishery_outer.
Checks item_id is a valid purchasable material.
Checks player has sufficient drabs.
Deducts cost from money_drabs.
Adds item to inventory (or increments quantity if already present).
Returns new state and an in-world message.

On wrong location:
 message: "Kilvin's students sell materials, but you're not
 in the Fishery."

On unknown item:
 message: "That's not something you can buy here."

On insufficient funds:
 message: "You haven't the coin for that."

Prices are defined as a constant object in this file.
Do not hardcode prices inline — define MATERIAL_PRICES once.

```
parseSympathyCommand(input: string):  
  SympathyAttempt | null
```

Parses player input into a SympathyAttempt.

Recognise these patterns:

- "use sympathy"
 -> SympathyAttempt with source = first candle in inventory,
 target = 'candle_tallow' (same type — simplest valid attempt)

intent = 'basic binding'

"bind [source] to [target]"

- > parse source and target as item ids or partial names
- match against player inventory items (case-insensitive)
- if source not found in inventory, return null

"use sympathy to [intent]"

- > attempt a basic binding with stated intent
- source = first suitable material in inventory
- target = same type if available, otherwise 'candle_tallow'
- intent = the stated text

Return null if no pattern matches.

NARRATION — src/content/sympathyNarration.ts

Export SYMPATHY_NARRATION: Record<string, string[]>

Each key maps to a pool of exactly 3 short narration strings.

Chosen deterministically: state.sympathy_state.times_used_today % 3

Keys required:

'success'
'slip'
'bleedthrough'
'backlash'
'no_source_material'
'invalid_target'
'alar_broken'
'too_cold'
'blocked_general'

Rules for all text:

- Original prose only. Do NOT reproduce Rothfuss lines.
- Second person present tense.
- For success: brief, precise, unsentimental. The binding works. Something moves, transfers, or changes. The cold is real.
- For slip: the link fails to hold. Nothing happens. The wasted effort is felt. Do not over-dramatise.
- For bleedthrough: the player feels what the bound object would feel. Brief, strange, physical. A hand that is not your hand.
- For backlash: pain. Short. No heroics. This is an injury.
- For blocked states: in-world, practical, no game jargon.
- Do not explain the laws of sympathy in the narration.
The player experiences the result. They can learn the theory

elsewhere.

- Maximum 2 sentences per entry.

ACTIONS — update src/engine/actions.ts

Add these commands to dispatch():

use sympathy / bind [x] to [y] / use sympathy to [intent]

- > call parseSympathyCommand(input)
- > if null: return narrator.renderFallback()
- > call adjudicate(attempt, state)
- > call applySympathyResult(state, result)
- > select narration from SYMPATHY_NARRATION using result.narration_key and times_used_today % 3
- > if result.injury is not null, append injury line to output
- > if new warmth < 30, append a cold warning line
- > return combined output string and newState

buy [item] / buy [quantity] [item]

- > parse item name and optional quantity (default 1)
- > call buyMaterial(item_id, quantity, state)
- > return result message and newState

sympathy status / alar status

- > return a brief formatted readout:
 - Alar strength: [label]
 - Warmth: [label]
 - Times used today: [n]
 - Active bindings: [n]
- Use descriptive labels not raw numbers:
 - alar: 80-100=sharp, 50-79=steady, 30-49=strained, 10-29=fragile, 0-9=broken
 - warmth: 80-100=warm, 60-79=comfortable, 40-59=cool, 20-39=cold, 0-19=dangerously cold

After every sympathy action, call checkTuitionDeadline() as per Prompt 2.

RENDERSTATUS UPDATE

Update renderStatus() in src/narration/renderStatus.ts to include:

Alar: [descriptive label from above]

Warmth: [descriptive label from above]

Only show these two lines. Do not show raw numbers.
Use the same label functions as sympathy status command.

Add and export:

```
alarLabel(alar_strength: number): string  
warmthLabel(warmth: number): string
```

Using the ranges defined above.

TIME ADVANCEMENT UPDATE

Update advanceTime() in src/engine/time.ts:

Each time step, warmth recovers slightly if not recently used:

```
if sympathy_state.times_used_today === 0:  
    warmth += 3 (max 100)  
else:  
    warmth += 1 (max 100)
```

Alar strength also recovers slightly per time step:

```
alar_strength += 4 (max 100)
```

On sleep (full night — 2 steps through night to morning):

```
alar_strength recovers to 80 minimum  
warmth recovers to 75 minimum  
times_used_today resets to 0
```

These recovery values are modest. They should not make sympathy feel consequence-free. They reflect rest and natural recovery only.

Modify the sleep() function in economy.ts to reset sympathy_state.times_used_today to 0 and apply the floor recoveries above when sleep succeeds.

TESTS — tests/sympathyEngine.test.ts

Import adjudicate, getSimilarityScore, calculateHeatCost, calculateRisk, resolveOutcome, applySympatbyResult, buyMaterial from src/engine/sympathyEngine.ts.

Import initDefaultPlayerState from src/engine/state.ts.

Use initDefaultPlayerState() as base for all tests.

Modify state fields inline per test — do not mutate the base object.

Tests:

getSimilarityScore:

1. getSimilarityScore('iron_drab', 'iron_drab') === 1.0
2. getSimilarityScore('candle_tallow', 'wax_stub') === 0.65
3. getSimilarityScore('iron_drab', 'clay_lump') === 0.15

calculateHeatCost:

4. High similarity (0.9) produces lower cost than low (0.1)
5. Weak alar (strength=30) produces higher cost than strong (70)

calculateRisk:

6. Cold state (warmth=20) produces higher risk than warm (warmth=80)
7. High fatigue (90) produces higher risk than low (10)

resolveOutcome (deterministic):

8. Same state always produces the same outcome
9. A state with risk=5 produces 'success' (very low risk)

adjudicate:

10. Attempt with source not in inventory returns
outcome: 'blocked', narration_key: 'no_source_material'
11. Attempt with valid materials and healthy state returns
outcome: 'success' (given default state values)
12. Attempt with warmth=5 returns outcome: 'blocked',
narration_key: 'too_cold'
13. Successful attempt returns heat_cost > 0

applySympathyResult:

14. After applying a success result, state.warmth is lower
than before
15. After applying a backlash result, injuries array is non-empty

buyMaterial:

16. Buying candle_tallow at fishery deducts correct drabs
17. Buying at wrong location returns failure message
18. Buying with insufficient funds returns failure message

ARCHITECTURE REMINDERS

- sympathyEngine.ts is engine-layer — may read PlayerState, must not call narrator functions or query the DB directly
- sympathyNarration.ts is content-layer — text only, no logic
- actions.ts assembles the output string from engine results and content pools

- narration/* never mutates state
- repl.ts saves state once per turn
- No `any` types
- No external API calls
- No Math.random() anywhere — all outcomes are deterministic

SUCCESS CRITERIA

This prompt is complete when:

- `use sympathy` with candles in inventory produces a result (success, slip, or otherwise) with in-world narration
- `bind iron\_drab to iron\_drab` produces a stronger result than `bind candle\_tallow to clay\_lump`
- Using sympathy multiple times in one day increases risk
- Using sympathy when cold increases risk and reduces warmth further
- A backlash adds an injury to the player's injuries list
- `sympathy status` displays alar and warmth as descriptive labels
- `buy candle\_tallow` at the Fishery deducts drabs and adds to inventory
- `buy candle\_tallow` outside the Fishery returns an in-world refusal
- `status` now shows alar and warmth labels
- Sleep recovers alar and warmth and resets times_used_today
- All 18 new tests pass alongside tests from Prompts 1 and 2
- No network calls at any point
- All outcomes are deterministic — same state, same result

=====
END OF PROMPT 3
=====